

MEMORIA TÉCNICA

Proyecto Final — Sistema de Gestión de Liga de eSports



Alumno: Raúl Sal Romeo

Curso: 1º DAW — Programación

1. Descripción General del Proyecto

El proyecto consiste en una aplicación Java para gestionar de forma integral una liga de eSports basada en la Liga de Videojuegos Profesional (LVP) de League of Legends.

El diseño sigue los principios de la Programación Orientada a Objetos (POO) aplicando herencia, clases abstractas, interfaces, polimorfismo, manejo de excepciones personalizadas y estructuras dinámicas.

2. Estructura del Proyecto

Todo el proyecto se organiza en un único paquete denominado Sal_Romeo_Raul_Proyecto_Final. A continuación se listan todos los archivos que lo componen:

Archivo	Tipo	Función principal
Main.java	Principal	Menú interactivo con do-while y switch-case
Liga.java	Contenedora	Gestiona equipos, partidos, cola FIFO, pila LIFO y calendario
Equipo.java	Modelo	Array fijo de 5 titulares, lista de suplentes y entrenadores
PersonaLiga.java	Abstracta	Clase base para Jugador y Entrenador
Jugador.java	Modelo	Extiende PersonaLiga, implementa Entrenable y Comparable
Entrenador.java	Modelo	Extiende PersonaLiga, implementa Entrenable y Comparable
Partido.java	Modelo	Representa un encuentro entre dos equipos
Incidencia.java	Modelo	Registra sanciones y eventos de la liga
Temporada.java	Modelo	Simula partidos y genera clasificación
ServicioEstadisticas.java	Utilidad	Métodos estáticos para rankings y estadísticas
Validador.java	Utilidad	Validaciones de edad, salario, email, nombre, etc.
Entrenable.java	Interfaz	Define entrenar() y calcularRendimiento()

Excepciones personalizadas incluidas en el proyecto:

Excepción	Cuándo se lanza
DatoInvalidoException	Dato de entrada inválido (edad, salario, email, nombre)
EquipoNoEncontradoException	Se busca o elimina un equipo que no existe
JugadorSancionadoException	Se intenta alinear como titular a un jugador sancionado
NombreDuplicadoException	Nickname o nombre de equipo ya existente
PresupuestoExcedidoException	El fichaje supera el presupuesto disponible del equipo
RolNoDisponibleException	El rol ya está ocupado en los titulares

3. Clases

3.1 PersonaLiga — Clase Abstracta

Es la clase base del sistema. Al ser abstracta, no puede instanciarse directamente; su función es definir la estructura común para Jugador y Entrenador.

Atributos:

- identificador — identificador único de la persona
- nombre — nombre real
- nickname — alias en la liga
- edad — entre 16 y 60 años
- salarioBase — salario mensual base en euros
- email — correo electrónico con formato válido

Métodos abstractos que cada subclase implementa de forma distinta:

- calcularCosteMensual() — calcula el coste mensual total con bonificaciones
- mostrarResumen() — muestra la información formateada de la persona

3.2 Jugador — Subclase de PersonaLiga

Representa a un jugador de la liga. Además de los atributos heredados, añade sus propios atributos de rendimiento y estado.

Atributos propios:

- rol — uno de los 5 roles: TOP, JUNGLE, MID, ADC, SUPPORT
- nivelMecanicas — habilidad mecánica (1-100)
- nivelEstrategia — capacidad táctica (1-100)
- partidasJugadas — experiencia acumulada
- mvpTotales — número de partidos como jugador más valioso
- sancion — indica si está inhabilitado para jugar

Métodos destacados:

- `entrenar()` — incrementa `nivelMecanicas` y `nivelEstrategia` de forma aleatoria (1-5 puntos), con tope en 100
- `calcularRendimiento()` — fórmula ponderada basada en sus estadísticas
- `calcularCosteMensual()` — `salarioBase` más bonus de 200€ por cada MVP
- `getPrecioFichaje()` — valor de mercado del jugador
- `compareTo()` — permite ordenar jugadores por rendimiento de mayor a menor

Fórmula de rendimiento:

```
calcularRendimiento() = (nivelMecanicas × 0.4)
                      + (nivelEstrategia × 0.3)
                      + (partidasJugadas × 0.02)
                      + (mvpTotales × 2)
```

3.3 Entrenador — Subclase de `PersonaLiga`

Representa al entrenador de un equipo. Su rendimiento depende de la experiencia y los resultados obtenidos.

Atributos propios:

- `añosExperiencia` — años ejerciendo como entrenador
- `especialidad` — área de especialización táctica
- `victoriasTotales` — número de victorias acumuladas

Fórmula de rendimiento:

```
calcularRendimiento() = (añosExperiencia × 2) + (victoriasTotales × 1.5)
```

`calcularCosteMensual() = salarioBase + (victoriasTotales × 150€)`

3.4 Equipo

Representa a un equipo de la liga. Combina una estructura estática (array de titulares) con una dinámica (lista de suplentes).

Atributos principales:

- `titulares` — `Jugador[5]`: array fijo con los 5 jugadores titulares
- `suplentes` — `ArrayList<Jugador>`: lista dinámica de jugadores de reserva
- `entrenadores` — `ArrayList<Entrenador>`: lista de entrenadores del equipo
- presupuesto, victorias, derrotas, empates, puntos

Operaciones que permite:

- Añadir/eliminar titulares y suplentes con validaciones
- Promover suplentes a titulares cuando hay un hueco libre
- Calcular el coste total de la plantilla
- Calcular el rendimiento medio del equipo

3.5 Liga

Clase contenedora principal del sistema. Gestiona todos los elementos: equipos, partidos, incidencias, el calendario, la cola de partidos y el historial de acciones.

- equipos — `ArrayList<Equipo>`: todos los equipos registrados
- colaPartidos — `LinkedList<Partido>`: cola FIFO de partidos por disputar
- historialAcciones — `ArrayList<String>`: pila LIFO de acciones realizadas
- calendario — `String[][]`: matriz bidimensional de jornadas
- mercado — `ArrayList<Jugador>`: jugadores disponibles para fichar
- incidencias — `ArrayList<Incidencia>`: registro de incidencias y sanciones

3.6 Partido

Representa un enfrentamiento entre dos equipos. Contiene identificador, jornada, equipos local y visitante, resultado, jugador MVP y estado (disputado o no).

3.7 Incidencia

Registra cualquier evento relevante de la liga: sanciones, expulsiones, aplazamientos u otros. Se almacena en una lista dinámica en Liga y permite búsqueda por equipo o jugador.

3.8 ServicioEstadisticas

Clase de utilidad con métodos estáticos para generar rankings: top jugadores por rendimiento o MVPs, mejor entrenador, equipo más valioso, clasificación ordenada, etc.

3.9 Validador

Clase de utilidad con métodos estáticos que validan todos los datos de entrada antes de crear o modificar objetos. Lanza `DatoInvalidoException` cuando un dato no cumple las reglas.

3.10 Entrenable — Interfaz

Interfaz propia del proyecto que deben implementar tanto Jugador como Entrenador. Define dos métodos:

- `void entrenar()` — lógica de mejora del rendimiento
- `double calcularRendimiento()` — fórmula numérica de evaluación

4. Justificación del Uso de Estructuras Dinámicas

4.1 Array Fijo — Titulares Jugador[5]

Cada equipo tiene exactamente 5 posiciones de juego (TOP, JUNGLE, MID, ADC, SUPPORT), por lo que un array de tamaño fijo es la estructura más adecuada. Se usa para almacenar la alineación, validar roles únicos, calcular el rendimiento del equipo y simular partidos. No usar un array aquí sería ignorar la restricción real del juego.

4.2 ArrayList — Suplentes y Entrenadores

El número de suplentes y entrenadores de un equipo no es fijo y puede variar a lo largo de la temporada. ArrayList permite añadir, eliminar y recorrer elementos sin límite predefinido, adaptándose a los cambios dinámicos del equipo.

4.3 Matriz Bidimensional — Calendario

Se usa una matriz `String[nJornadas][nEnfrentamientos]` para representar el calendario de la temporada. Las filas son las jornadas y las columnas los enfrentamientos de cada jornada. Implementa el algoritmo round-robin para generar todos los partidos posibles entre los equipos.

4.4 Cola FIFO — Partidos Pendientes

Los partidos que aún no se han disputado se almacenan en una cola (comportamiento FIFO). El siguiente partido en disputarse siempre es el primero que se encoló. Esto simula de forma realista el orden de celebración de los partidos a lo largo de la temporada.

```
encolarPartido(partido)      // añade al final
disputarSiguientePartido()   // extrae del principio y simula resultado
mostrarSiguientePartido()    // muestra sin extraer
vaciarCola()                 // limpia todos los partidos pendientes
```

4.5 Pila LIFO — Historial de Acciones

Cada operación relevante del usuario (crear equipo, fichar jugador, registrar resultado, aplicar sanción...) se registra en una pila con marca temporal. Al mostrar el historial se recorre de fin a inicio, mostrando primero la acción más reciente. También permite deshacer la última acción.

```
registrarAccion(descripcion) // añade al final del ArrayList
mostrarHistorial()           // recorre de fin a inicio (LIFO)
deshacerUltimaAccion()       // elimina el último elemento
```

4.6 HashMap — Clasificación

La clasificación se genera con un `HashMap<Equipo, Integer>` que asocia cada equipo con sus puntos. Se ordena después con `Collections.sort()` y el `Comparator POR_PUNTOS` definido en `Equipo`.

4.7 HashSet — Control de Duplicados

Se usa `HashSet` para controlar que no haya equipos o identificadores duplicados en la liga, aprovechando que la búsqueda en un `HashSet` es $O(1)$.

5. Excepciones Personalizadas

Todas las excepciones del proyecto extienden `Exception` (excepciones chequeadas), lo que obliga a declararlas con `throws` o capturarlas con `try-catch` en cada punto de uso.

Excepción	Cuándo se lanza	Ejemplo de mensaje
DatoInvalidoException	Dato de entrada no válido	El campo edad tiene un valor inválido: 15
EquipoNoEncontradoException	Equipo que no existe	No se encontró ningún equipo con el nombre XYZ
JugadorSancionadoException	Titular sancionado	El jugador LuisX está sancionado y no puede jugar
NombreDuplicadoException	Nombre ya existente	Ya existe un equipo con el nombre KOI
PresupuestoExcedidoException	Sin presupuesto suficiente	Presupuesto: 1.000€, Coste: 1.500€
RolNoDisponibleException	Rol ya ocupado	El rol TOP ya está ocupado por DragonSlayer

Flujo típico de error:

1. El usuario introduce un dato inválido
2. Validador.validarXxx() lanza DatoInvalidoException
3. El setter de PersonaLiga captura y muestra el mensaje
4. Main captura la excepción y continúa el flujo del menú

6. Menú de la Aplicación

La aplicación se organiza en un menú principal con 3 opciones y múltiples submenús:

Opción	Submenús disponibles
1. Gestión	1.1 Personas (alta, listar, buscar, modificar, eliminar) 1.2 Equipos (crear, eliminar, ver, convocatoria) 1.3 Fichajes (fichar, vender, promover, sustituir)
2. Datos	2.1 Calendario · 2.2 Cola de partidos FIFO · 2.3 Partidos 2.4 Incidencias y sanciones · 2.5 Clasificación 2.6 Estadísticas · 2.7 Historial (pila LIFO) · 2.8 Deshacer
3. Salir	Con confirmación antes de cerrar la aplicación

7. Datos de Ejemplo

La aplicación se inicializa con 5 equipos reales de la LVP, cada uno con 5 jugadores titulares, 2 suplentes y 2 entrenadores:

Equipo	Ciudad	Presupuesto	Titulares	Suplentes	Entrenadores
KOI	Madrid	200.000€	5	2	2
MAD Lions	Barcelona	190.000€	5	2	2
Team Heretics	Valencia	180.000€	5	2	2
Fnatic	Sevilla	170.000€	5	2	2

Equipo	Ciudad	Presupuesto	Titulares	Suplentes	Entrenadores
Giants	Málaga	160.000€	5	2	3

Además se incluyen 5 jugadores y 3 entrenadores en el mercado libre disponibles para ser fichados por cualquier equipo.

8. Conclusión

Este es mi proyecto de fin de año de Programación, la realización del mismo ha sido una experiencia que me ha puesto los pies de la tierra. Me ha gustado la sensación del proyecto, además el dedicarle tiempo no ha sido algo que se haya sentido pesado. Agradecer al profesor y tutor Javier Seijas Bellido por las enseñanzas de este años, haciendo que la asignatura con más peso del grado ha hecho que sea fácil.